Java Section 5

MORE ON METHODS AND VARIABLES

You will learn more details about methods and variables, and also review examples to reenforce much of the learning to date.

Java Section 5
More about Methods

- Java Programs and Classes
- Static methods
- Static variables
- Method calls
- Method overloading
- Variable-length argument lists
- Command-line arguments
- JOptionDialog
- Wrapper Classes
- Case studies

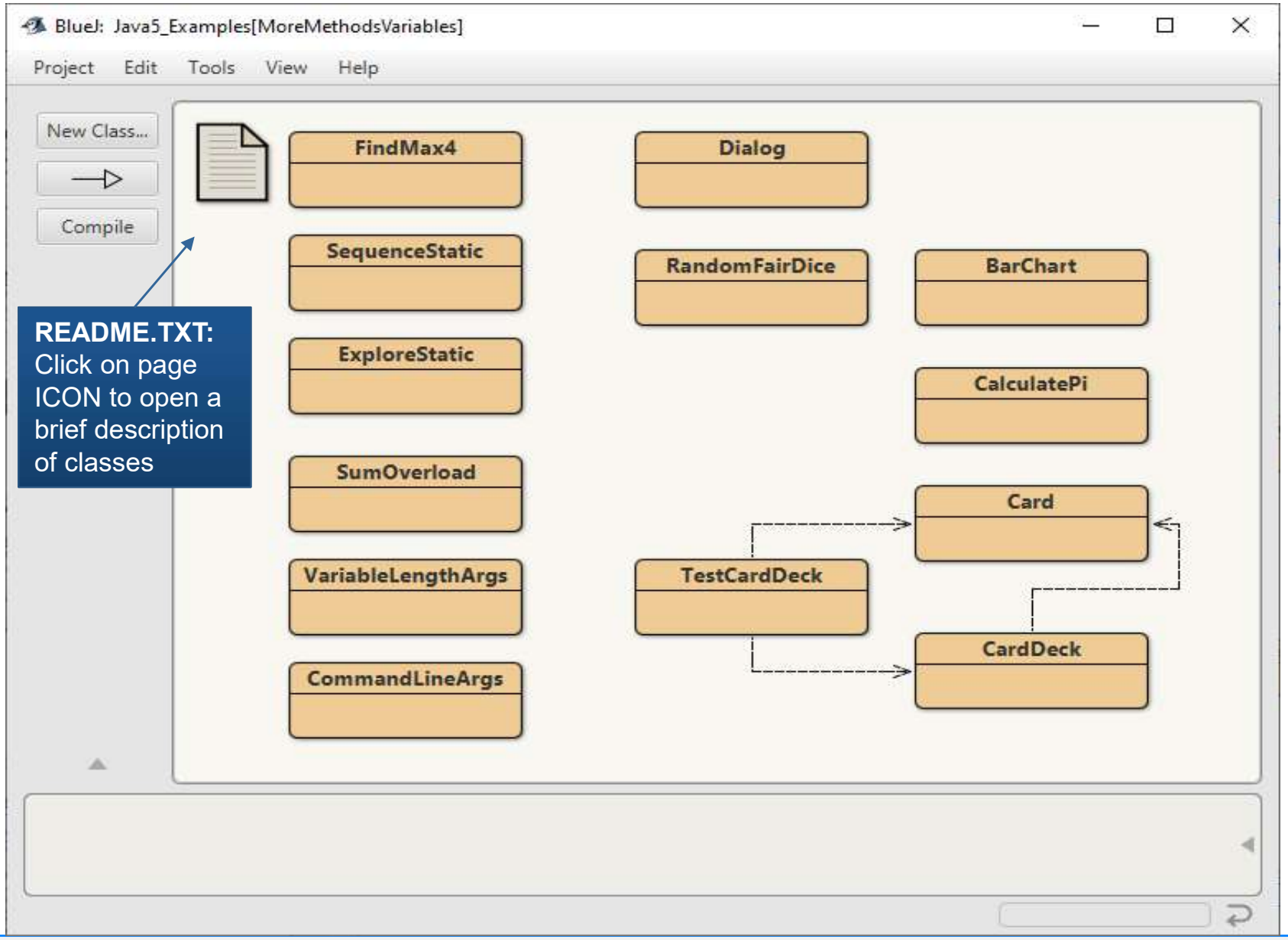
Lectures will emphasize the most important!

It is strongly recommended that you also read the recommending readings and complete the associated exercises.

JN

BlueJ: Java5_Examples[MoreMethodsVariables]

Examples discussed in this section and more: Go Explore!

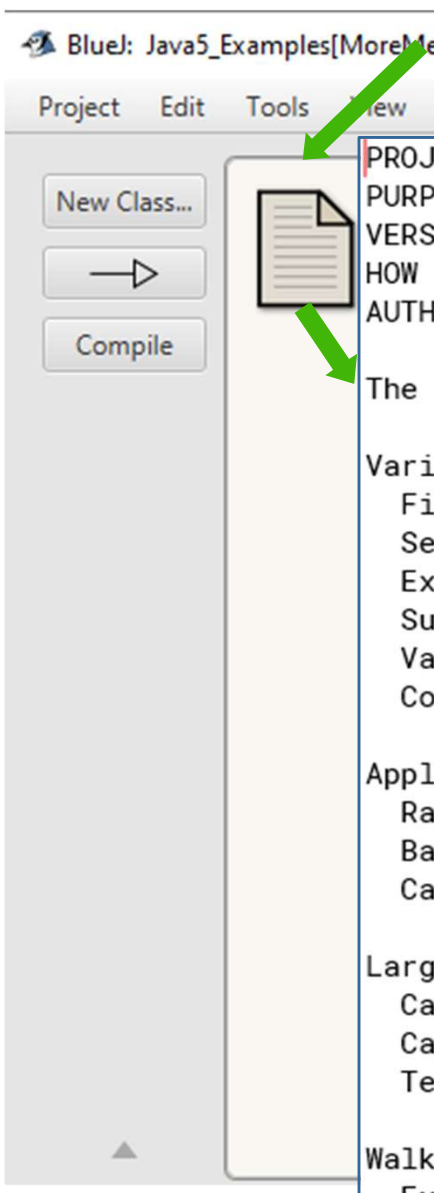


Note:
Normally classes would be grouped in separate projects.

The classes are collected here for ease of opening and comparison.

BlueJ: Java5_Examples[MoreMethodsVariables]

Examples discussed in this section: Go Explore!



README.TXT: Click on page ICON to open a brief description

```
PROJECT TITLE:              Java5_Examples[MoreMethodsVariables]
PURPOSE OF PROJECT:         More Examples of Methods and Variables
VERSION or DATE:            AY25
HOW TO START THIS PROJECT:  Open the project and explore the classes.
AUTHORS:                    John

The examples illustrate and explore methods, variables and classes.

Various Method Examples
  FindMax4              Demonstrate a static method and calling via class name.
  SequenceStatic        Display a sequence to illustrate static methods and a static variable.
  ExploreStatic         Explore static variables and methods based on MyMath methods.
  SumOverload           Demonstrate method overloading and type conversion.
  VariableLengthArgs    Demonstrate variable length argument lists using a min method.
  CommandLineArgs       Demonstrate command line argument lists by displaying the args array.

Application case studies demonstrating concepts covered to date.
  RandomFairDice        Roll a computer dice and check if fair.
  BarChart              Display a simple bar chart using asterisks.
  CalculatePi           Calculate Pi using Gregory-Leibniz series.

Larger application, basic text based card deck
  Card                  Represent a playing card with a face and suit.
  CardDeck              Represent a deck of French-suited playing cards with deal and shuffle.
  TestCardDeck          Test our deck of cards.

Walkthrough code to give deeper insight. Explore using a debugger.
  ExploreMethods        Demonstrate argument passing to methods, static variables and methods.
```

Java Programs and Classes

(more details in Java2 and Java5)

- The challenge is to break a large problem through **divide and conquer** into relatively small and simple building blocks.
- The building blocks of **Object Oriented (OO) Systems** are **classes**.
  - Java programs are made up of **classes** which contain **fields** and **methods**.
    - The **fields** maintain features or properties and are stored in variables.
    - The **methods** implement the tasks to be completed (the behaviour) using statements and variables.
  - The design and implementation effort is to construct the solution from existing and new building blocks (components).
- **OO Systems** are developed through several iterative ***life-cycle*** stages of:
  - **Specification**: identify and document the ***Requirements*** and ***Use Cases***
  - **Design**: specify the classes and detailed application behaviour
  - **Code and Test**: the new classes (***unit test***) and the application (***system test***).
- **Software reusability**
  - Use existing well-tested components where possible. Create reluctantly!

Instance Variables and Instance Methods
Recall

- An **instance variable** is used to store an attribute of an object.

```
private String name;
private int quantity;
```

- An **object** is an instance of a class.
- Each created object has its own set of instance variables.
- An **instance method** normally accesses/modifies instance variables.
  - Get methods to access e.g. **getName**(), **getQuantity**()
  - Set methods to modify e.g. **setName**(), **setQuantity**()
- An instance method is called through an object of its class.

objectReference.methodName(arguments)

static Methods

- When a method does not depend on any instance variables, it is known as a **class method** or **static method**.
  - The **static** keyword is used to identify them, hence known as **static methods.**
  - As **static methods** don't use instance variables, we do not have to create an object of the class (using **new**) to call a **static method.**
- Classes may contain **static methods** to perform general tasks, e.g.
  - **Java class Math** methods to implement mathematical formulae
  - **Java class Arrays** methods to perform tasks on arrays.
- To declare place keyword **static** before the return type.

public static returnType methodName (parameters)

- To call a static method

methodName(arguments) if called from in same class

ClassName.methodName(arguments) if called in a different class then prefix with the **ClassName**

Static Methods
max of 4 real numbers

FindMax4.java

```
/**
 * Calculate the maximum of four double parameters
 * @param a   the first parameter
 * @param b   the second parameter
 * @param c   the third parameter
 * @param d   the fourth parameter
 * @return the maximum of the four parameters
 */
public static double max( double a, double b, double c, double d )
{
   double maximum = a;   // start by assuming a is the maximum
   if ( b > maximum )
      maximum = b;
   if ( c > maximum )
      maximum = c;
   if ( d > maximum )
      maximum = d;
   return maximum;
}
```

To call the static max method:

max(1.2, 3.4, 5.6, 7.8) // if called from method in same class

FindMax4.max(1.2, 3.4, 5.6, 7.8) // if called from different class

static Methods
The main method

- The main method is always declared as static

```
public static void main ( String[] args )
{
}
```

- When you run an application e.g., with the **java** command, the Java Virtual Machine (**JVM**) locates the **main** method.
  - As it is static, the **JVM** can call it without creating an instance of the class
  - Hence, initially no object creation is required.
- When a **main** method is executed:
  - **main** can accept one parameter, an array of type **String** called **args**
  - The array elements contain 0 or more **command line arguments (CLAs)**
  - **CLAs** are specified when starting the application.

static variables
(aka class variables and static fields)

- Previously we met **instance variables** where each object (instance) of the class has its own instance of the variable.
- A class Module representing a university module with a title attribute would declare an instance variable:

```
public class Module {
    private String title;  // instance variable, one for each module created
```

- **static variables,** also known as **class variables**
  - are declared for variables of a class which **do not** require a separate instance of the variable for each object.
  - All objects of a class containing **static variables** share one copy.
- Assuming all modules of class Module are in the same university

```
private static String university;
```

- Together the **static (class) variables** and **instance variables** represent the **fields** of a class.

static variable

count number of times a method is called

SequenceStatic.java

```
public class SequenceStatic
{
    private static int countToSeqStringCalls = 0;

    /**
     * Generate a String representing the given sequence
     * @param a        first number in sequence
     * @param n        number of terms in sequence
     * @param d        the common difference to the previous term
     * @return         String with space separate elements enclosed in <>
     */
    public static String toSeqString(int a, int n, int d)
    {
        ++countToSeqStringCalls;      // static/class variable

        String seqString = "<";
        for ( int term = 0; term < n; term++ )
        {
            seqString += String.format("%3d ", a+d*term );
        }
        return seqString + ">";
    }
}
```

Use a static variable to count the number of times a method has been called.

Method generates a String representation of an arithmetic sequence

< a a+d a+2d ... a+(n-1)d >

static variable

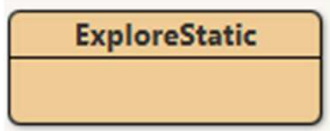
count number of times a method is called

SequenceStatic.java

```
public static void main(String[] args)
{
    System.out.printf("a=%2d, n=%2d, d=%2d %s\n", 0, 4, 2, toSeqString(0, 4, 2));
    System.out.printf("a=%2d, n=%2d, d=%2d %s\n", 0, 4,-2, toSeqString(0, 4, -2));

    // toSeqString returns a String containing sequence
    String sequenceA = toSeqString(1, 4, 3);
    System.out.println(sequenceA);

    // Invoke toSeqString via class name SequenceStatic
    String sequenceB = SequenceStatic.toSeqString(10, 6, 5);
    System.out.println(sequenceB);

    // Invoke toSeqString via object reference to SequenceStatic object.
    SequenceStatic sequenceObject = new SequenceStatic();
    String sequenceC = sequenceObject.toSeqString(-10, 6, -5);
    System.out.println(sequenceC);

    // Display how many sequence Strings were created.
    System.out.printf("\nNumber of sequence Strings %d\n",
        countToSeqStringCalls );
}
```

```
a= 0, n= 4, d= 2 <  0   2   4   6 >
a= 0, n= 4, d=-2 <  0  -2  -4  -6 >
<  1   4   7  10 >
< 10  15  20  25  30  35 >
<-10 -15 -20 -25 -30 -35 >

Number of sequence Strings 5
```

static methods and variables



ExploreStatic.java

- class **ExploreStatic** provides a collection of static methods from the MyMath challenge.
  - factorial
  - biCoeff
  - power
    - uses a **for** loop
  - pow
    - uses **Math.pow()**
- Demonstrates
  - argument passing to methods
  - static (aka class) variables
  - static (aka class) methods

```
factorial(6) is 720
biCoeff( 4, 2) i.e.  4 choose 2 is  6
biCoeff( 6, 3) i.e.  6 choose 3 is 20
biCoeff(24, 2) i.e. 24 choose 2 is  3???

Different calls to power methods for 2.0 to power of 4
16.0
16.0
16.0
The power and pow methods differ by 1.35525e-20!
     We have a   power(0.1,4)=0.00010000000000000003
     difference!   pow(0.1,4)=0.00010000000000000002

factorial() was called 14 times
```

Java class Math

- class **Math** provides a collection of static methods that enable you to perform common mathematical calculations.
- Many of the methods are overloaded to support different primitive types
- Two math constants for mathematical constants π and e

  Math.PI 3.141592653589793
  Math.E 2.718281828459045

- Are declared in class **Math** with the modifier keywords as follows:

  ```
  public static final double E = 2.718281828459045;
  public static final double PI = 3.141592653589793;
  ```

  - Modifier public allows you to use these fields in other classes.
  - Modifier static ensures one version per class
  - Modifier final declares the field as constant

- class **Math** is in the **java.lang** package so you don't need to import it.

Java class Math
Example methods

- To call a method, prefix with the Math class name, see Example column.

Math.methodName(arguments)

Method	Description	Example
abs(a)	**Absolute** value of a	Math.abs(-12.2) is 12.2
cos(a)	Trigonometric **cosine** of a (radians)	Math.cos(Math.PI) is 1.0
exp(a)	**Exponential** e^a	Math.exp(1) is 2.7182818…
log(a)	Natural **logarithm** of a	Math.log(Math.E) is 1.0
min(a,b)	**smaller** of a and b	Math.min(1.5, 2.4) is 1.5
max(a,b)	**greater** of a and b	Math.min(1.5, 2.4) is 2.4
pow(a,b)	a raised to the **power** b (a^b)	Math.pow(2.0, 16.0) is 65536
sin(a)	Trigonometric **sine** of a (radians)	Math.sin(MathPI/4) is 0.7071…
sqrt(a)	**Square root** of a	Math.sqrt(625.0) is 25.0
tan(a)	Trigonometric **tangent** of a (radians)	Math.tan(MathPI/4) is 1.0

Methods

Declaration, formal parameters and actual arguments.

- Each method must be declared within a class.
- A method cannot be declared inside another method.
- The parameters for a static or non-static method, are given in the method declaration and enclosed in **( )**.

  public static returnType methodName (parameters)

  public returnType methodName (parameters)

- Multiple parameters are specified in a comma separated list**.**
- Each parameter is known as a **formal parameter** to the method.
- When calling a method, there must be an **actual argument** for each formal parameter.

  Note: The terms **parameter** and **argument** are often used interchangeably.

Methods
Three ways of calling

1. Using a method name by itself to call another method of the same class

   methodName(arguments)

2. Using a variable that contains a reference to an object, followed by a dot **.** and the method name to call any **method** of the referenced object

   objectReference.methodName(arguments)

3. Using the class name and a dot (.) to call a **static method** of a class

   ClassName.methodName(arguments)

- An instance (non-static) method can call any method of the same class directly and can manipulate any of the class's fields directly.
- A static method can directly call only other static methods and directly manipulate only static fields of the same class.

Methods
Returning from a method

- **Recall: methods must be declared within the body of a class.**

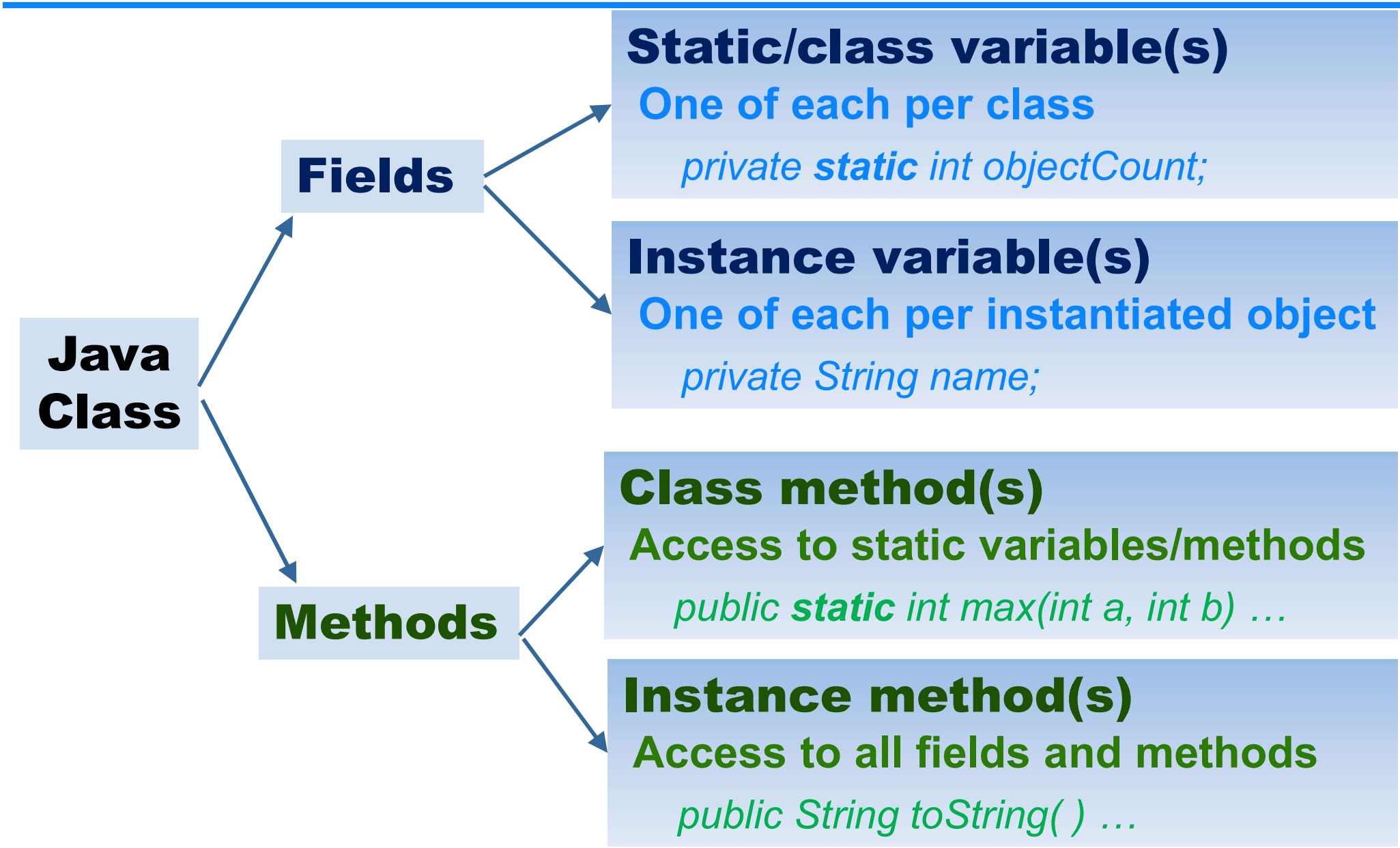
```
public class ClassName
{
    return-value-type methodName( method-parameters )
    {
        // Statements to realise the method behaviour
        return return-value-expression;
    }
}
```

- **Optional method-parameters**

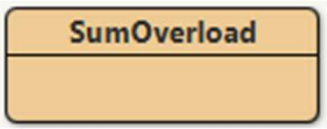
  return return-value; required if the return-value-type is not void

- **Three ways to return control to the statement that calls a method:**
  1. When the program flow reaches the method-ending right brace **}**
  2. Using just return **;** applies only when **return-value-type** is **void**
  3. When the method returns a result: return **return-value-expression ;**

static vs instance (non-static) fields and methods
Summary



Methods
Overloading



SumOverload.java

- Methods can have the same name if they have different parameter lists
  - e.g. **sum** to find the sum of the parameter values

```
public static int sum( int a, int b, int c )
public static double sum( double a, double b, double c )
```

- The compiler will select the appropriate method by comparing the number of parameters and their types. The return type is ignored.

```
int sum1 = sum( 4, 8, 12 );          // will select first sum above
double sum2 = sum( 4.2, 5.6, 7.2 );  // will select second sum above
double sum3 = sum( 4.2, 5, 7 );      // will select second sum above
```

- Overloaded methods can have different numbers of parameters
- If the compiler cannot resolve, it will generate an error message.
- Recall: the challenge **MyMath** int and double **min** methods were overloaded.

Argument Promotion

- **Argument promotion** among primitive types takes place to ensure that a method receives its corresponding parameter where possible.
- Java has a set of promotion rules to specify allowed conversions
  - Applied when expression operators have more than one primitive type,
  - And when passing primitive-type values arguments to a method.
- Values will be promoted when necessary to the “highest” type .

Primitive Type	Allowed promotions
double	none
float	double
long	double float
int	double float long
char	double float long int
short	double float long int
byte	double float long int short
boolean	none

Examples

1. *Method* **Math.cos(4)**

Since Math.cos() requires a parameter of type double, the int 4 is promoted to double 4.0 before calling.

2. *Expression:* **4.2 * 5**

becomes 4.2*5.0 before multiplication, where int 5 is promoted to double 5.0

Casting
Explicit conversion

- Converting values to types from higher to lower in the table could result in loss, and hence is not implicitly done (compilation error)
  - Converting a floating-point value to integer could lose the fractional part.
  - Converting a long to an integer may result in integer overflow

(**type**) **value** will cast value to type (aka type convert)

Primitive Type	Allowed promotions
double	none
float	double
long	double float
int	double float long
char	double float long int
short	double float long int
byte	double float long int short
boolean	none

Examples

1. int result = (int) (3.1+2.8);

result is 5 as (int) truncates *(floors or rounds towards 0).*

2. (int) (Math.sqrt(x) + 0.5);

Math.sqrt() returns double hence above will truncate to the nearest integer.

Passing Arguments to Methods

Consider a MyMath average method

```
public class MyMathTest
{
    public static void main(String[ ] args)
    {
        double result = MyMath.average( 1.8,  2,  7.2/2 + 4,  3*2 - 1 );
    }
}

public class  MyMath
{
    public static double average(double a, double b, double c, double d)
    {
        return (a+b+c+d)/4;
    }
}
```

1. Call MyMath.**average** where formal parameters a, b, c and d become, after argument promotion:

double **4.1**

double **a** 1.8 | double **b** 2.0 | double **c** 7.6 | double **d** 5.0

2. **average** method then executes and evaluates the expression

return (1.8+2.0+7.6+5.0)/4;

i.e. return double value 4.1

3. The **average** method **return value** will then be assigned to the local variable **result**

Methods
Scope of Declarations

- We declare entities such as variables and methods.
- The **scope** of a declaration, when it said to be "**in scope**", is the part of a class or method that can refer to the entity.
- Each block of statements enclosed in { } can have its own variable declarations.
- Local variables and parameter names can hide instance or class variables while **"in scope"**, known as shadowing.
- The **scope** of a parameter or local variable
  - Scope of a parameter is the entire method body.
  - Scope of a local variable is from its declaration to the end of that block.
  - Scope of a variable declared in a **for** statement initialisation, is the entire **for**.
- The **scope** of a method or field is the entire class body.
  - Except in static methods which can only access other static methods.

Variable length argument lists

- Variable length argument lists are used to create methods that can receive an unspecified number of arguments
- The last parameter **type** is followed by an ellipsis (…)
  - To indicate the variable number of arguments of that **type** can follow
  - The ellipsis can only occur in the last parameter.
- The system will create an array of **type** to hold the parameters

public static int **min**(int **…** numbers)

declares a method which can accept any number of parameters of type int.

Variable length argument lists

VariableLengthArgs.java

```
/**
 * Find the minimum of the variable number of arguments
 * @param number arguments to find the minimum
 * @return the minimum (if no argument return Integer.MAX_VALUE)
 */
public static int min(int ... numbers)
{
    int min = Integer.MAX_VALUE;
    for( int number : numbers)
        if (number < min)
            min = number;
    return min;
}
```

```
public static void main( String[] args )
{
    System.out.printf( "min(3,2,1) is %d\n", min(3, 2, 1) );
    System.out.printf( "min(3,2)   is %d\n", min(3, 2) );
    System.out.printf( "min(3)     is %d\n", min(3) );
    System.out.printf( "min()      is %d\n", min() );
}
```

```
min(3,2,1) is 1
min(3,2)   is 2
min(3)     is 3
min()      is 2147483647
```

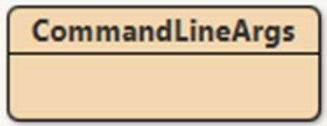
Command line arguments

- Arguments can be passed to an application from an Operating System (OS) command line, or in an IDE.
  - In OS : **java MyApplication** argA argB "argC in quotes"
  - In IDE, there is usually a run option where they can be specified.
  - They are separated by white space
- In Java, these argument are presented to the main method as an array of Strings

  public static void main(**String[] args**)
- The number of arguments is **args.length**
- **args[0]** is argA, **args[1]** is argB, **args[2]** is "argC in quotes"
- Print using **for (String arg : args)**
  System.out.println(arg);

Command line arguments

CLAs



CommandLineArgs.java

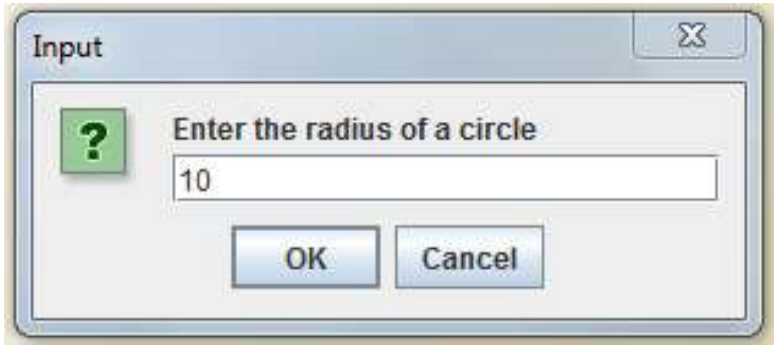
```
public class CommandLineArgs
{
    public static void main( String[] args )
    {
        if (args.length == 0)
            System.out.println("There are no command line arguments");

        for (String arg : args)
            System.out.println(arg);

        for (int i = 0; i < args.length; i++)
            System.out.printf( "args[%d] = %s\n", i, args[i] );
    }
}
```

- Use the command line **java** command to run the application, with CLAs

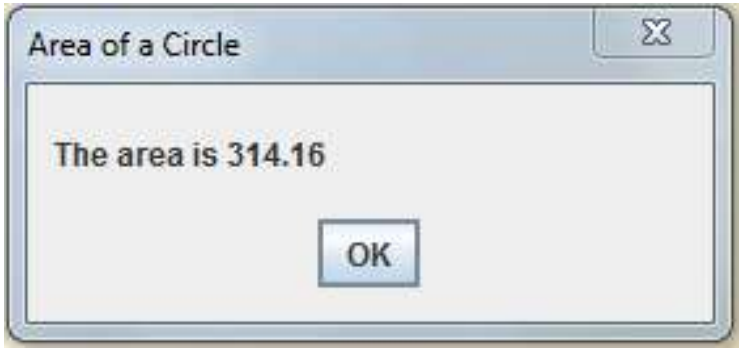
```
>java CommandLineArgs -h -v "Hello there"
-h
-v
Hello there
args[0] = -h
args[1] = -v
args[2] = Hello there
```

Graphical User Interface (GUI) using JOptionPane

- Graphical User Interfaces (GUI) improve user interaction
- GUI Applications use windows or dialog boxes (also called dialogs) to interact with the user.
- Many applications can be developed using two simple GUI dialog boxes for inputting and outputting information.
  - available in the class **JOptionPane** in package **javax.swing**.
- The JOptionPane dialogs input and output a String
  - The input String is typically converted to another type e.g. a primitive such as int or double



Input Dialog

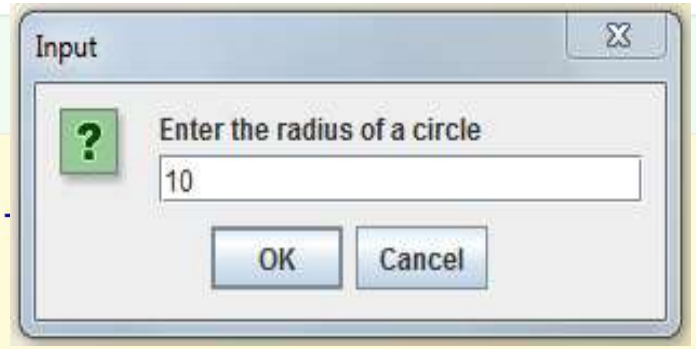


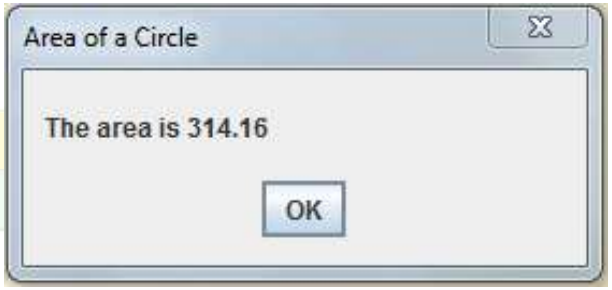
Message Dialog

Example
JOptionPane Dialogs

Dialog.java

```
import javax.swing.JOptionPane;
public class Dialog
{
    /**
     * Using dialogs display the area of a circle
     */
    public static void main(String[] args)
    {
        String radiusString =
            JOptionPane.showInputDialog( "Enter the radius of a circle" );

        double radius = Double.parseDouble( radiusString ); // Convert to double
        double area = Math.PI*radius*radius;

        String areaString = String.format("The area is %.2f", area);
        JOptionPane.showMessageDialog( null, areaString,
            "Area of a Circle", JOptionPane.PLAIN_MESSAGE );
    }
}
```





JOptionPane.showInputDialog conversion to a primitive type

- **JOptionPane.showInputDialog** always returns a **String**
- Must convert from **String** to the required primitive type
- Typically use the Wrapper class method **parse***Type*.

Primitive Type	Conversion method
int	**Integer.parseInt**(String s)
double	**Double.parseDouble**(String s)
float	**Float.parseFloat**(String s)
boolean	**Boolean.parseBoolean**(String s)
char	Use the String **charAt**(int index) method
byte	**Byte.parseByte**(String s)
short	**Short.parseShort**(String s)
long	**Long.parseLong**(String s)

Java Primitive Wrapper Classes

- A Java primitive wrapper class is one of eight classes provided to provide object methods for the primitive types
- An extensive set of methods are provided, see Java API
- Available in the **java.lang** package so no need to import

Primitive Type	Wrapper Class	Constructor Arguments
int	Integer	int, String
double	Double	double, String
float	Float	float, String
boolean	Boolean	boolean, String
char	Character	char
byte	Byte	byte, String
short	Short	short, string
long	Long	long, String

JOptionPane Exercise

- Using the code from the Dialog.java class, modify to find the volume of a cone of radius r and height h, which are input by the user using a **showInputDialog**.
- Display the volume V using a **showMessageDialog**.

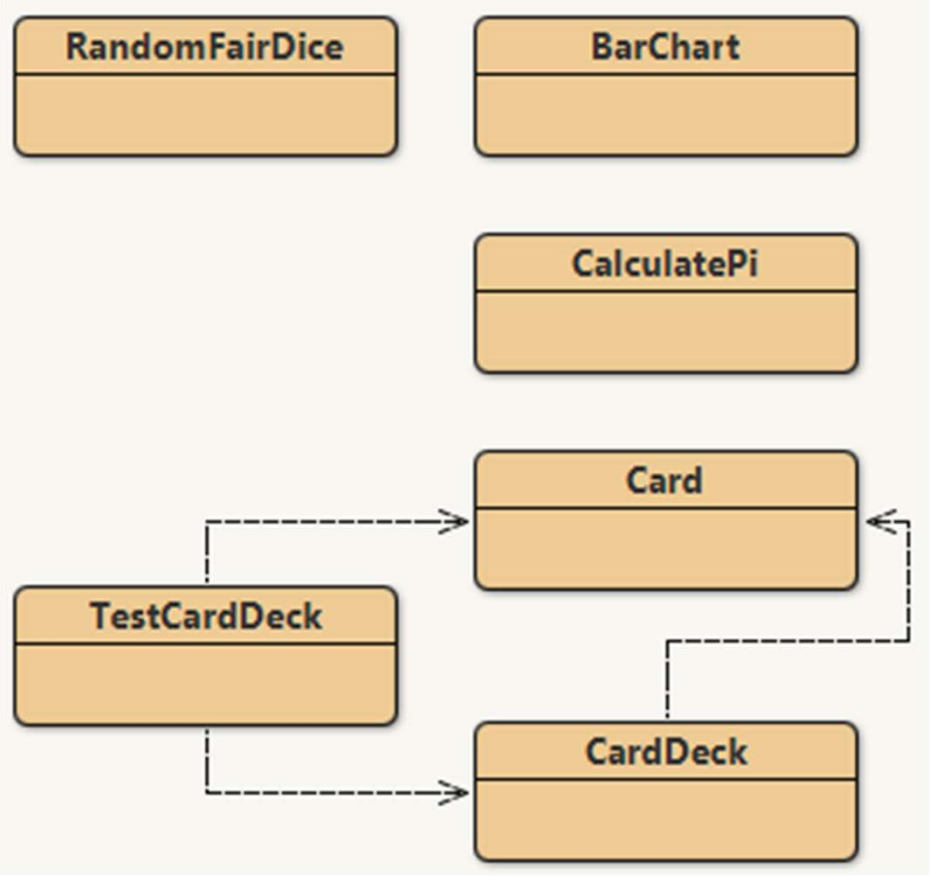
$$V = \frac{1}{3} \pi r^2 h$$

- Explore the **JOptionPane class in the Java API**
- Explore the various methods e.g.

  showMessageDialog,
  showInputDialog,
  showConfirmDialog,
  showOptionDialog

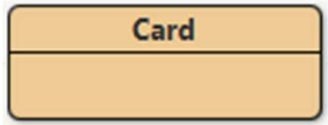
Application Case Studies

- You are encourage to explore the source code of these classes to get more familiar with source code and Java concepts.
- Start by running the code and observe the output
- Then read the code
- Modify and extend the code
  - Start by duplicating the class so you have a backup nearby if necessary.
  - In **BlueJ**, right mouse click on class icon and select **Duplicate**



Deck of Cards Case Study

TestCardDeck, CardDeck, Card



Card.java

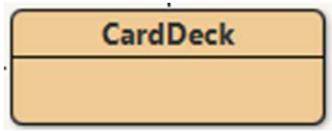
```
public class Card
{
   private String face;
   private String suit;

   /**
    * Construct a card with face and suit
    */
   public Card( String face, String suit )
   {
      this.face = face;
      this.suit = suit;
   }

   public String toString()
   {
      return "Card:{face:" + face + ", suit:" + suit + "}";
   }

   public String toCard()
   {
      return face + " of " + suit;
   }
}
```

```
The lucky card is Eight of Spades

                <<< Shuffled Deck of Cards #1 with 52 cards >>>

| Eight of Spades   | King of Spades    | Queen of Hearts   | Deuce of Spades  |
| Deuce of Clubs    | Five of Clubs     | Six of Diamonds   | Deuce of Diamonds|
| Deuce of Hearts   | Jack of Diamonds  | Ten of Spades     | King of Clubs    |
| Six of Spades     | Ace of Diamonds   | King of Hearts    | Eight of Diamonds|
| Three of Clubs    | Jack of Hearts    | Queen of Spades   | Ten of Diamonds  |
| Six of Clubs      | Ace of Spades     | Three of Diamonds | Five of Spades   |
| King of Diamonds  | Five of Hearts    | Ace of Hearts     | Four of Clubs    |
| Jack of Spades    | Ten of Clubs      | Three of Hearts   | Nine of Spades   |
| Six of Hearts     | Four of Hearts    | Nine of Clubs     | Nine of Hearts   |
| Five of Diamonds  | Ace of Clubs      | Eight of Hearts   | Four of Diamonds |
| Seven of Hearts   | Ten of Hearts     | Queen of Diamonds | Nine of Diamonds |
| Seven of Clubs    | Four of Spades    | Jack of Clubs     | Eight of Clubs   |
| Three of Spades   | Queen of Clubs    | Seven of Diamonds | Seven of Spades  |
```

```
        <<< Unshuffled Deck of Cards #2 with 54 cards with Jokers >>>

| Ace of Hearts     | Ace of Clubs      | Ace of Diamonds   | Ace of Spades    |
| Deuce of Hearts   | Deuce of Clubs    | Deuce of Diamonds | Deuce of Spades  |
| Three of Hearts   | Three of Clubs    | Three of Diamonds | Three of Spades  |
| Four of Hearts    | Four of Clubs     | Four of Diamonds  | Four of Spades   |
| Five of Hearts    | Five of Clubs     | Five of Diamonds  | Five of Spades   |
| Six of Hearts     | Six of Clubs      | Six of Diamonds   | Six of Spades    |
| Seven of Hearts   | Seven of Clubs    | Seven of Diamonds | Seven of Spades  |
| Eight of Hearts   | Eight of Clubs    | Eight of Diamonds | Eight of Spades  |
| Nine of Hearts    | Nine of Clubs     | Nine of Diamonds  | Nine of Spades   |
| Ten of Hearts     | Ten of Clubs      | Ten of Diamonds   | Ten of Spades    |
| Jack of Hearts    | Jack of Clubs     | Jack of Diamonds  | Jack of Spades   |
| Queen of Hearts   | Queen of Clubs    | Queen of Diamonds | Queen of Spades  |
| King of Hearts    | King of Clubs     | King of Diamonds  | King of Spades   |
| Joker of Red      | Joker of Black
```

Deck of Cards

Basic Card Shuffler: Random select/swap



CardDeck.java

- The class CardDeck creates decks of standard cards, extended decks, supports: shuffling, dealing, peeking and displaying, and more

```
/**
 * Shuffle deck of cards using a random number generator
 * For each card, select another random card and swap
 * @return   the shuffled carddeck
 */
public CardDeck shuffle()
{
    Random randomNumbers = new Random();

    for ( int card = 0; card < deck.length; card++ )
    {
        int secondCard =  randomNumbers.nextInt( STANDARD_DECK_LENGTH );
        Card tempCard = deck[ card ];
        deck[ card ] = deck[ secondCard ];
        deck[ secondCard ] = tempCard;
    }
    isShuffled = true;
    currentCard = 0;      // reinitialize currentCard to first card
    return this;
}
```

This is the code for a basic shuffle operation.

Do explore all the classes:

Card
CardDeck
TestCardDeck

Improvements welcome!

static and instance methods and variables
Exercise: bringing it all together

- Create a class Student
- Instance variable and methods
  - Add a single instance variable called **name**, of appropriate type.
  - Code a set and get method for the name instance var.
- Static variable and method
  - Add a static/class variable **count** to maintain a count of the number of Student objects constructed.
  - Create a method **getCount** that returns the value of **count**
- Create a no argument Student constructor that increments **count** each time it is called.
- Execute and explore

Next Semester

More skills, confidence, work, freedom, fun...

- Explore more Java API classes
- Enumerated Types
- Exceptions
- Inheritance
- Polymorphism
- Java GUIs
- Java Graphics
- Strings and Regular Expressions
- Recursion
- Searching and Sorting

+ using a Professional IDE

And there's still more afterwards!

- *Generic Collections*
- *Lambdas and Streams*
- *Concurrency*
- *Database Access*
- *Mobile Apps*
- *Enterprise Apps*
- *Networking/Cloud*

+++

Signing off here, and wishing you all the best in life's journey, with or without Java!
John